Lecture - 8 : Byte pair Encoding

Tokenization algorithms

1. Word Based
2. Sub-word Based
3. Character Based

My hobby is playing cricket

'My' 'hobby' 'is' 'playing' 'cricket'

Problem: what do we do with out of vocabulary (OOV) words, different meaning of similar words [boy, boys]

My hobby is playing cricket

['m', 'y', 'h'....]

Solve the OOV problems

- very small vocabulary. Every language has fixed number of characters (english ≈ 256)

Problems: The meaning associated with words is completely lost. Also the tokenized sequence is much larger that the initial raw text.

Sub-word Tokenizer

Rule 1.: when you get data set. Don't split frequently words into smaller subwords.

***

Rule 2.: split the rare words into smaller meaniful subwords.

"boy" should not be split.

"boys" should be split into "boy" & "s".

Advantages of Sub-word Tokenization

1. The subword splitting helps the model learn that different words with same root word as "token" like "tokens" and "tokenizing" are similar in meaning.

2. It also helps the model learn that "tokenization" and "modernization" are made of different root words but have the same suffix "ization" and are used in same syntactic situations.

Byte Pair Encoding (BPE)

It is a subword *** tokenization algorithms.

BPE algorithms (1994) : Most common pair of consecutive byte of data is replaced with a byte that does not occur in data.

3. Let us take a simple example to understand this (source: wiki)

Original data: aaabdaaabac

(a.) The byte pair "aa" occurs the most. We will replace it with Z as Z does not occur in the data.

(b.) compressed data: ZabdZabac

(c.) The next common byte pair is "ab". We will replace this by Y.

(d.) compressed data: ZYdZYac

any byte pair that appears only once, so we do not create it.

4. How is the BPE algorithm used for LLMs???

(a.) BPE ensures that most common words in the vocabulary are represented as a single token while rare words are broken down into two or more subwords tokens.

(b) Let us take a practical example:

* Let us consider the below dataset of words:

{"old": 7, "older": 3, "finest": 9, "lowest": 4}

* Preprocessing: we need to add end token "</w>" at the end of each word.

{"old</w>": 7, "older</w>": 3, "finest</w>": 9, "lowest</w>": 4}

* Let us now split words into character and count their freq.

Number	Token	frequency
1	</w>	23
2	o	14
3	l	14
4	d	10
5	e	16
6	r	3
7	f	9
8	i	9
9	n	9
10	s	13
11	t	13
12	w	4

(c) Next step in the BPE algorithm is to look for the most frequent pairing.

merge them and perform the same iteration again and again until we reach the token limit or iteration limit.

(d) Iteration-1: Start with second most common, take "e" most common byte pair starting with 'e'. "es".

Number	Token	frequency
1	</w>	23
2	o	14
3	l	14
4	d	10
5	e	16-13=3
6	r	3
7	f	9
8	i	9
9	n	9
10	s	13-13=0
11	t	13
12	w	4
13	es	9+4=13

(e) Iteration-2: Merge the tokens "es" and "t" as they have appeared 13 times in our dataset.

Number	Token	frequency
1	</w>	23
2	o	14
3	l	14
4	d	10
5	e	16-13=3
6	r	3
7	f	9
8	i	9
9	n	9
10	s	13-13=0
11	t	13
12	w	4
13	es	9+4=13-13=0
14	est	13-13=0
15	est</w>	13

helps algorithm difference now to estimate and highest.

(g) Iteration-4: "O" and "l" has appeared 10 times.

Number	Token	frequency
1	</w>	23
2	o	14-10=4
3	l	14-10=4
4	d	10
5	e	16-13=3
6	r	3
7	f	9
8	i	9
9	n	9
10	s	13-13=0
11	t	13
12	w	4
13	es	9+4=13-13=0
14	est	13-13=0
15	ol	7+3=10

(h) Iteration-5: "ol" and "d" has appeared 10 times.

Number	Token	frequency
1	</w>	23
2	o	14-10=4
3	l	14-10=4
4	d	10-10=0
5	e	16-13=3
6	r	3
7	f	9
8	i	9
9	n	9
10	s	13-13=0
11	t	13-13=0
12	w	4
13	es	9+4=13-13=0
14	est	13-13=0
15	est</w>	13
16	ol	7+3=10
17	old	7+3=10

(i) "f", "i", "n" appear 9 times. But we just have one word with these character. So we are not merging them.

(j) Let us remove tokens with zero count.

Number	Token	frequency
1	</w>	23
2	o	14-10=4
3	l	14-10=4
4	e	16-13=3
5	r	3
6	f	9
7	i	9
8	n	9
9	w	4
10	est</w>	13
10	old	10

This list of 11 tokens will serve as our vocabulary.

The stopping criteria can either be token count or the number of iterations.